

Minishell — Mandatory Part Task List

Scope: mandatory only, per subject v10.0 · Team of 2 · Target: 20–25 days

0. Project Setup & Rules (shared, do together first)

- Create the Git repository; only what is in the repo gets graded.
- Makefile with rules: **\$(NAME)**, **all**, **clean**, **fclean**, **re** — compiled with **cc -Wall -Wextra -Werror**, no relinking. Program name: **minishell**.
- Copy libft sources + its Makefile into a **libft/** folder; your Makefile builds libft first, then the project.
- Agree on the **shared data structure**: a list of commands, each holding its argv and its list of redirections. This is the parser-to-executor contract.
- All code must follow the Norm; no crashes (segfault, double free), no memory leaks in your own code (readline's internal leaks are tolerated).
- Only allowed external functions (readline family, fork/execve/wait family, dup/dup2/pipe, open/close/read/write, signal/sigaction, getcwd/chdir, getenv, stat family, etc. — see subject table).

1. Input Loop & History

- Main loop: display a **prompt**, read a line with **readline()**.
- Add non-empty lines to **history** with **add_history()**.
- Handle EOF from readline (ctrl-D) → exit the shell cleanly.

2. Lexer / Tokenizer

- Split the input line into tokens: words, |, <, >, <<, >>.
- Detect **unclosed quotes** → error, do not interpret.
- Do NOT interpret unrequired special characters: \ (backslash) and ; (semicolon) — no special handling needed.

3. Quote Handling

- **Single quotes** '...': everything inside is literal — no expansion at all.
- **Double quotes** "...": everything literal EXCEPT \$ (variables still expand).
- Remove the quote characters themselves from the final arguments.

4. Expansion

- Expand **\$VAR** (dollar followed by characters) to its environment value; unknown variables expand to empty.
- Expand **\$?** to the exit status of the last executed foreground pipeline.
- Respect quote rules: no expansion inside single quotes.

5. Parser (build the command list)

- Turn tokens into the shared structure: commands separated by pipes, each with argv + redirections in order.
- Basic syntax error handling (e.g. pipe or redirection with no target), with a clean error message — take bash as reference.

6. Redirections

- **<** redirect input from a file.
- **>** redirect output to a file (truncate).
- **>>** redirect output in append mode.
- **<< DELIM** (heredoc): read lines until the delimiter is seen, feed as input; does not need to go into history.
- Handle `open()` errors (no such file, permission denied) like bash: print error, fail that command.

7. Pipes

- Implement **|**: connect stdout of each command to stdin of the next via `pipe()` + `dup2()`.
- Fork one child per command in the pipeline; parent closes unused fds and waits for all children.
- Exit status of the pipeline = exit status of the **last** command.

8. Executor

- If command contains a **/**, treat as relative/absolute path; otherwise search each directory in **PATH**.
- `fork()` + `execve()` for external commands; `waitpid()` and convert the child's status into **\$?**.
- Errors like bash: 'command not found' → exit status 127; permission denied → 126.
- Builtins run without fork when alone (so `cd/export/exit` affect the shell itself); inside a pipeline they run in the child.

9. Built-in Commands

- **echo** with option `-n`.
- **cd** with only a relative or absolute path.
- **pwd** with no options.
- **export** with no options (add/update environment variables).
- **unset** with no options (remove variables).
- **env** with no options or arguments (print environment).
- **exit** with no options (quit the shell, set exit status).
- You'll need your own editable copy of the environment (`getenv` alone isn't enough for `export/unset`).

10. Signals

- In interactive mode: **ctrl-C** → new prompt on a new line; **ctrl-D** → exits the shell; **ctrl-** → does nothing.
- Use **at most one global variable**, storing ONLY the signal number — no structs, no data access from the handler.
- Signal behavior must match bash (also while a child command is running and during heredoc).

11. Memory, Errors & Testing

- Free everything you allocate; run `valgrind` regularly (suppress only `readline`'s own leaks).
- No segfaults / bus errors / double frees anywhere — that's an automatic 0.
- Write your own quick test scripts (not submitted, not graded) and compare every behavior against bash.
- When unsure about ANY behavior: open bash, try it, copy it.

12. README.md (required deliverable)

- First line, italicized: *This project has been created as part of the 42 curriculum by <login1>, <login2>.*
- **Description** section: what the project is, its goal, brief overview.
- **Instructions** section: how to compile and run.
- **Resources** section: references used AND a description of how AI was used (which tasks, which parts).
- Written in English.

Suggested Split for Two People

Person A — Parsing side	Person B — Execution side
Input loop & history (1) Lexer (2) Quotes (3) Expansion (4) Parser (5)	Builtins (9) Executor + PATH (8) Redirections (6) Pipes (7) Signals (10)
Shared: setup (0), memory/testing (11), README (12), integration & debugging together.	

Rough 20–25 Day Plan

- **Days 1–3:** setup, Makefile, libft, design + agree on the shared command structure.
- **Days 4–12:** build in parallel (A: lexer→quotes→expansion→parser · B: builtins→executor→redirections→pipes).
- **Days 13–18:** integrate the two halves, add signals, fix behavior mismatches vs bash.
- **Days 19–25:** edge cases, heredoc corner cases, valgrind, Norm cleanup, README, mock defense with each other.